

IHC – Artificial Intelligence **for Real-World Application (AI)**

Modular Assignment No. 1

Intelligent Campus Assistant – Path Optimisation System

Name : Abhinav M

SRN : 31250837

FY M.sc Statistics – Data Science

Topic: Search and Reasoning

Academic Year: 2025–2026

VISHWAKARMA UNIVERSITY, PUNE

Topic	Search and Reasoning
Module	Modular Assignment No. 1
Algorithm	BFS & A* with Dynamic Reasoning
Language	Python 3

1. Problem Analysis

1.1 Problem Statement

Students and faculty at university campuses regularly face difficulties navigating efficiently, particularly during peak hours and campus events. Conventional maps and navigation apps lack the ability to dynamically account for real-world factors such as crowd density, ongoing events, or adverse weather conditions.

This project designs and implements an AI-powered Campus Assistant that uses search and reasoning algorithms to recommend the most efficient paths between campus locations — adapting in real-time to changing environmental conditions.

1.2 Stakeholders

- Students navigating between classes, hostels, and facilities
- Faculty members moving between academic blocks
- Visitors and new students unfamiliar with the campus layout
- Campus administration monitoring crowd flow

1.3 Key Challenges

- Static maps do not reflect real-time crowd or event data
- Multiple simultaneous conditions can compound navigation difficulty
- Shortest-distance path is not always the fastest or safest path
- Weather conditions make certain routes undesirable or hazardous

1.4 Goals & Success Criteria

- Find optimal routes using informed search algorithms (A*)
- Dynamically adjust path costs based on crowd, weather, and events
- Demonstrate improvement over naive (BFS) routing under adverse conditions
- Provide clear, human-readable path recommendations

2. Search Space and Constraints

2.1 Graph Representation

The campus is modelled as a weighted undirected graph where:

- Nodes represent campus locations (10 nodes)
- Edges represent walkable paths between locations
- Edge weights represent effective traversal cost (distance × condition multipliers)

2.2 Campus Nodes

Location	X Coordinate	Y Coordinate
Main Gate	0	0
Library	2	3
Cafeteria	4	1
Admin Block	2	6
Science Block	5	5
Sports Complex	7	2
Hostel A	0	6
Hostel B	1	8
Auditorium	4	8
Parking Lot	7	0

2.3 Constraints

- Crowd density > 70%: path weight multiplied by 1.8×
- Crowd density 40–70%: path weight multiplied by 1.3×
- Raining + outdoor path: weight multiplied by 1.5×
- Active event at adjacent venue: weight multiplied by 2.0×
- Covered/indoor paths are immune to weather penalties

2.4 Heuristic Function

A* uses the Euclidean straight-line distance between any node and the goal as its admissible heuristic:

Heuristic $h(n)$

$h(n) = \sqrt{(x_{goal} - x_n)^2 + (y_{goal} - y_n)^2}$ This is admissible because straight-line distance never overestimates actual path cost.

3. Algorithm Implementation

3.1 Breadth-First Search (BFS)

BFS explores all neighbouring nodes level by level, guaranteeing the path with the fewest hops (edges). It treats all edges as equal weight and ignores real-world conditions — serving as a baseline for comparison.

Time Complexity: $O(V + E)$ | Space Complexity: $O(V)$

BFS — Core Logic

```
queue = deque([(start, [start])]) while queue:     node, path =
queue.popleft()     for neighbour in graph[node]:         if neighbour not
in visited:             if neighbour == goal: return path
queue.append((neighbour, path + [neighbour]))
```

3.2 A* Search (Primary Algorithm)

A* is an informed best-first search that combines the actual cost from the start (g-score) with a heuristic estimate to the goal (h-score). It always finds the optimal path when the heuristic is admissible and consistent.

Time Complexity: $O(E \log V)$ | Space Complexity: $O(V)$

A* — Core Logic

```
open_set = [(h(start, goal), 0.0, start, [start])] while open_set:     f,
g, node, path = heappop(open_set)     if node == goal: return path, g
for neighbour, cost in graph[node].items():         new_g = g + cost
new_f = new_g + h(neighbour, goal)         heappush(open_set, (new_f,
new_g, neighbour, ...))
```

3.3 Algorithm Comparison

Property	BFS	A*
Optimality (hops)	Yes	Not always
Optimality (cost)	No	Yes (admissible h)
Uses weights	No	Yes
Real-world aware	No	Yes
Time Complexity	$O(V+E)$	$O(E \log V)$
Best used for	Hop minimisation	Cost minimisation

4. Reasoning Logic for Dynamic Path Adjustment

4.1 Overview

The Reasoning Engine intercepts the static graph before each query and applies a set of logical rules to dynamically adjust edge weights. This transforms the Campus Assistant from a static path-finder to a context-aware intelligent system.

4.2 Rule Set

Condition	Trigger	Effect on Weight
Crowd (high)	Density > 70%	×1.8 (avoid)
Crowd (moderate)	Density 40–70%	×1.3 (slow)
Rain (outdoor)	Raining + no cover	×1.5 (prefer indoor)
Event nearby	Active event at venue	×2.0 (major congestion)
Indoor path	Any rain condition	No penalty (safe)

4.3 Decision Flow

Reasoning Pipeline
1. Collect real-time inputs: crowd[], is_raining, active_events[] 2. For each edge (A → B):
 a. Start with base distance b. Apply crowd multiplier (avg of endpoints) c. Apply rain multiplier (if outdoor segment) d. Apply event multiplier (if edge is adjacent to event venue)
3. Rebuild graph with updated weights 4. Run A* on updated graph → return optimal path

5. Simulations and Examples

Scenario 1 — Normal Day

Route: Main Gate → Hostel B | **Conditions:** None

BFS and A* both selected the same 4-hop route through Library → Admin Block → Hostel A → Hostel B. With no adverse conditions, the unweighted and weighted graphs produce identical results, confirming baseline correctness.

Algorithm	Path Found
BFS	Main Gate → Library → Admin Block → Hostel A → Hostel B (4 hops)
A*	Main Gate → Library → Admin Block → Hostel A → Hostel B (cost: 10.90)

Scenario 2 — Rainy Weather

Route: Main Gate → Science Block | **Conditions:** Raining

Outdoor paths received a 1.5× penalty. The algorithm correctly preferred the route through the Library (a partially covered corridor), reducing exposure to rain. Both algorithms agreed since the covered indoor route was also the shortest.

Algorithm	Path Found
BFS	Main Gate → Library → Science Block (2 hops)
A*	Main Gate → Library → Science Block (cost: 10.80)

Scenario 3 — Auditorium Event (Evening Concert)

Route: Hostel A → Cafeteria | **Conditions:** Event at Auditorium, Crowd at Admin Block (70%), Auditorium (90%)

The event at the Auditorium caused its adjacent paths to be heavily penalised (×2.0). Additionally, the high crowd at Admin Block added further cost. A* routed away from the event zone successfully through Library → Cafeteria.

Algorithm	Path Found
BFS	Hostel A → Admin Block → Library → Cafeteria (3 hops)
A*	Hostel A → Admin Block → Library → Cafeteria (cost: 8.00)

Scenario 4 — Peak Hours (Key Differentiator)

Route: Parking Lot → Library | **Conditions:** High crowd at Main Gate (80%), Cafeteria (85%), Science Block (75%)

This scenario best demonstrates A*'s advantage. BFS chose the direct 2-hop route via Main Gate (which was heavily crowded at 80%), while A* intelligently re-routed through Sports

Complex → Science Block → Library — a longer path in hops, but significantly lower effective cost due to avoiding crowded nodes.

Algorithm	Path Found
BFS	Parking Lot → Main Gate → Library (2 hops) — passes through 80% crowd!
A*	Parking Lot → Sports Complex → Science Block → Library (cost: 8.60) — avoids crowd

5.5 Results Summary

Scenario	Route	BFS Hops	A* Cost
Normal Day	Main Gate → Hostel B	4	10.90
Rainy Weather	Main Gate → Science Block	2	10.80
Auditorium Event	Hostel A → Cafeteria	3	8.00
Peak Hours (Key Result)	Parking Lot → Library	2	8.60

6. Conclusion

This project successfully demonstrates how AI search algorithms, when combined with a dynamic reasoning engine, can significantly improve campus navigation beyond what static maps offer.

Key findings:

- A* consistently outperforms BFS in weighted/real-world scenarios by selecting paths with lower actual traversal cost rather than just fewer hops.
- The Reasoning Engine effectively encodes real-world knowledge (crowd, weather, events) as dynamic edge weights — making the system context-aware without requiring complex external data pipelines.
- Scenario 4 (Peak Hours) provides the clearest demonstration of intelligent rerouting: A* chose a 3-hop path over BFS's 2-hop path because the shorter path passed through dangerously congested areas.
- The system is extensible: new sensors, event APIs, or weather feeds can plug directly into the Reasoning Engine without modifying the search layer.

Future improvements could include real-time crowd sensor integration, time-of-day modelling, and a mobile app front-end for live navigation assistance.

7. Appendix

Python Code :

```
.....
```

Intelligent Campus Assistant - Path Optimisation System

IHC Modular Assignment No. 1 | Topic: Search and Reasoning

```
.....
```

```
import heapq
```

```
from collections import deque
```

```
import math
```

```
# _____
```

```
# 1. CAMPUS GRAPH DEFINITION
```

```
# _____
```

```
# Nodes: Campus locations
```

```
LOCATIONS = {
```

```
    "Main Gate":    (0, 0),
```

```
    "Library":      (2, 3),
```

```
    "Cafeteria":    (4, 1),
```

```
    "Admin Block":  (2, 6),
```

```

"Science Block": (5, 5),

"Sports Complex": (7, 2),

"Hostel A": (0, 6),

"Hostel B": (1, 8),

"Auditorium": (4, 8),

"Parking Lot": (7, 0),

}

# Edges: (location_a, location_b, base_distance)

EDGES = [

    ("Main Gate", "Library", 3.6),

    ("Main Gate", "Cafeteria", 4.1),

    ("Main Gate", "Parking Lot", 7.0),

    ("Library", "Admin Block", 3.0),

    ("Library", "Cafeteria", 2.8),

    ("Library", "Science Block", 3.6),

    ("Cafeteria", "Sports Complex", 3.2),

    ("Cafeteria", "Science Block", 4.1),

    ("Admin Block", "Hostel A", 2.2),

    ("Admin Block", "Auditorium", 2.8),

    ("Admin Block", "Science Block", 3.2),

```

```
("Science Block", "Auditorium", 3.6),  
("Science Block", "Sports Complex", 2.8),  
("Hostel A", "Hostel B", 2.1),  
("Hostel B", "Auditorium", 3.2),  
("Sports Complex", "Parking Lot", 2.2),  
]
```

```
# Build adjacency list
```

```
def build_graph(edges, weights=None):
```

```
    """Build adjacency list graph. weights override base distances."""
```

```
    graph = {loc: {} for loc in LOCATIONS}
```

```
    for a, b, base_dist in edges:
```

```
        cost = weights.get((a, b), weights.get((b, a), base_dist)) if weights else base_dist
```

```
        graph[a][b] = cost
```

```
        graph[b][a] = cost
```

```
    return graph
```

```
# _____
```

```
# 2. REASONING ENGINE — DYNAMIC WEIGHT ADJUSTER
```

```
# _____
```

```
class ReasoningEngine:
```

```
    """
```

```
    Applies real-world factors to edge weights:
```

- Crowd density : multiplies weight on crowded paths
- Weather : prefers covered/indoor routes when raining
- Events : blocks or penalises paths near active events

```
    """
```

```
# Covered/indoor path segments (safe from rain)
```

```
COVERED_PATHS = {
```

```
    ("Library", "Admin Block"),
```

```
    ("Admin Block", "Hostel A"),
```

```
    ("Admin Block", "Auditorium"),
```

```
    ("Science Block", "Auditorium"),
```

```
}
```

```
# Paths near event-prone venues
```

```
EVENT_ADJACENT = {
```

```
    "Auditorium": [("Science Block", "Auditorium"),
```

```
                    ("Hostel B", "Auditorium"),
```

```
                    ("Admin Block", "Auditorium")],
```

```

    "Sports Complex": [("Cafeteria", "Sports Complex"),

                        ("Science Block", "Sports Complex"),

                        ("Sports Complex", "Parking Lot")],

}

```

```

def __init__(self):

    self.crowd_levels = {loc: 0.0 for loc in LOCATIONS} # 0–1

    self.is_raining = False

    self.active_events = set() # venue names

```

```

def set_conditions(self, crowd_levels=None, is_raining=False, active_events=None):

    if crowd_levels:

        self.crowd_levels.update(crowd_levels)

    self.is_raining = is_raining

    self.active_events = set(active_events) if active_events else set()

```

```

def compute_weights(self):

    """Return edge-weight dictionary reflecting current conditions."""

    weights = {}

    for a, b, base in EDGES:

```

```

cost = base

# --- Crowd penalty ---

avg_crowd = (self.crowd_levels.get(a, 0) + self.crowd_levels.get(b, 0)) / 2

if avg_crowd > 0.7:

    cost *= 1.8 # heavy crowd → nearly double traversal time

elif avg_crowd > 0.4:

    cost *= 1.3 # moderate crowd


# --- Rain penalty (prefer covered paths) ---

if self.is_raining:

    edge = (a, b)

    rev = (b, a)

    if edge not in self.COVERED_PATHS and rev not in self.COVERED_PATHS:

        cost *= 1.5 # outdoor path less desirable in rain


# --- Event penalty ---

for venue, adjacent in self.EVENT_ADJACENT.items():

    if venue in self.active_events:

        if (a, b) in adjacent or (b, a) in adjacent:

            cost *= 2.0 # congestion around event venues

```

```
weights[(a, b)] = cost
```

```
weights[(b, a)] = cost
```

```
return weights
```

```
def describe_conditions(self):
```

```
    lines = []
```

```
    high_crowd = [loc for loc, v in self.crowd_levels.items() if v > 0.5]
```

```
    if high_crowd:
```

```
        lines.append(f" 🧑 High crowd at: {' '.join(high_crowd)}")
```

```
    if self.is_raining:
```

```
        lines.append(" ☔ Weather: Raining (outdoor paths penalised)")
```

```
    if self.active_events:
```

```
        lines.append(f" 🗣️ Active events at: {' '.join(self.active_events)}")
```

```
    if not lines:
```

```
        lines.append(" ⚙️ Normal conditions — no special factors")
```

```
    return "\n".join(lines)
```

```
# _____
```

```
# 3. BFS — SHORTEST HOPS (UNWEIGHTED)
```

#

```
def bfs(graph, start, goal):
```

```
    """Returns (path, hop_count) using BFS (fewest edges)."""
```

```
    if start == goal:
```

```
        return [start], 0
```

```
    visited = {start}
```

```
    queue = deque([(start, [start])])
```

```
    while queue:
```

```
        node, path = queue.popleft()
```

```
        for neighbour in graph[node]:
```

```
            if neighbour not in visited:
```

```
                new_path = path + [neighbour]
```

```
                if neighbour == goal:
```

```
                    return new_path, len(new_path) - 1
```

```
                visited.add(neighbour)
```

```
                queue.append((neighbour, new_path))
```

```
    return None, float('inf') # no path found
```



```
# _____  
  
# 4. A* — OPTIMAL WEIGHTED PATH  
  
# _____
```

```
def euclidean_heuristic(node, goal):
```

```
    """Straight-line distance between node and goal (admissible heuristic)."""
```

```
    x1, y1 = LOCATIONS[node]
```

```
    x2, y2 = LOCATIONS[goal]
```

```
    return math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
```

```
def astar(graph, start, goal):
```

```
    """Returns (path, total_cost) using A* with Euclidean heuristic."""
```

```
    if start == goal:
```

```
        return [start], 0.0
```

```
    # Priority queue: (f_score, g_score, node, path)
```

```
    open_set = [(euclidean_heuristic(start, goal), 0.0, start, [start])]
```

```
    visited = {} # node → best g_score seen
```

```
    while open_set:
```

```
f, g, node, path = heapq.heappop(open_set)
```

```
if node == goal:
```

```
    return path, round(g, 2)
```

```
if node in visited and visited[node] <= g:
```

```
    continue
```

```
visited[node] = g
```

```
for neighbour, edge_cost in graph[node].items():
```

```
    new_g = g + edge_cost
```

```
    if neighbour not in visited or visited[neighbour] > new_g:
```

```
        h = euclidean_heuristic(neighbour, goal)
```

```
        new_f = new_g + h
```

```
        heapq.heappush(open_set, (new_f, new_g, neighbour, path + [neighbour]))
```

```
return None, float('inf')
```

```
# _____
```

```
# 5. SIMULATION RUNNER
```

```
# _____
```

```
engine = ReasoningEngine()
```

```
def run_simulation(title, start, goal, crowd_levels=None,
```

```
    is_raining=False, active_events=None):
```

```
    print(f"\n{'='*60}")
```

```
    print(f" SCENARIO: {title}")
```

```
    print(f" Route  : {start} → {goal}")
```

```
    print(f"{'='*60}")
```

```
    # Apply conditions
```

```
    engine.set_conditions(
```

```
        crowd_levels = crowd_levels or {},
```

```
        is_raining    = is_raining,
```

```
        active_events = active_events or []
```

```
    )
```

```
    print("\nConditions:")
```

```
    print(engine.describe_conditions())
```

```
    # Build weighted graph
```

```
    weights = engine.compute_weights()
```

```
graph = build_graph(EDGES, weights)
```

```
# BFS (unweighted hops)
```

```
bfs_path, bfs_hops = bfs(build_graph(EDGES), start, goal) # always unweighted
```

```
# A* (weighted, condition-aware)
```

```
astar_path, astar_cost = astar(graph, start, goal)
```

```
print(f"\n💡 BFS Path ({bfs_hops} hops):")
```

```
print(f"  {' → '.join(bfs_path)}")
```

```
print(f"\n⭐ A* Path (cost = {astar_cost:.2f}):")
```

```
print(f"  {' → '.join(astar_path)}")
```

```
if bfs_path != astar_path:
```

```
    print("\n💡 Reasoning: A* chose a different route than BFS because")
```

```
    print("    real-world conditions increased the cost of the direct path.")
```

```
else:
```

```
    print("\n✅ Both algorithms agree on the optimal route.")
```

```
return {
```

```
"scenario" : title,  
  
"start"    : start,  
  
"goal"     : goal,  
  
"bfs_path" : bfs_path,  
  
"bfs_hops" : bfs_hops,  
  
"astar_path": astar_path,  
  
"astar_cost": astar_cost,  
  
}
```

```
# _____
```

```
# 6. SCENARIOS
```

```
# _____
```

```
results = []
```

```
results.append(run_simulation(
```

```
    title    = "Scenario 1 — Normal Day (No Special Conditions)",
```

```
    start    = "Main Gate",
```

```
    goal     = "Hostel B",
```

```
))
```

```
results.append(run_simulation(  
  
    title      = "Scenario 2 — Rainy Weather",  
  
    start      = "Main Gate",  
  
    goal       = "Science Block",  
  
    is_raining = True,  
  
))
```

```
results.append(run_simulation(  
  
    title      = "Scenario 3 — Auditorium Event (Evening Concert)",  
  
    start      = "Hostel A",  
  
    goal       = "Cafeteria",  
  
    active_events= ["Auditorium"],  
  
    crowd_levels = {"Auditorium": 0.9, "Admin Block": 0.7},  
  
))
```

```
results.append(run_simulation(  
  
    title      = "Scenario 4 — Peak Hours (Multiple Crowded Zones)",  
  
    start      = "Parking Lot",  
  
    goal       = "Library",  
  
    crowd_levels = {  
  
        "Cafeteria" : 0.85,
```

```
        "Main Gate" : 0.80,  
  
        "Science Block" : 0.75,  
  
    },  
  
))
```

```
print(f"\n{'='*60}")
```

```
print(" SUMMARY TABLE")
```

```
print(f"{'='*60}")
```

```
print(f"{'Scenario':<45} {'BFS Hops':>8} {'A* Cost':>8}")
```

```
print("-" * 63)
```

```
for r in results:
```

```
    short = r['scenario'].split('—')[1].strip()
```

```
    print(f"{'short':<45} {'bfs_hops':>8} {'astar_cost':>8.2f}")
```

